模式之模式

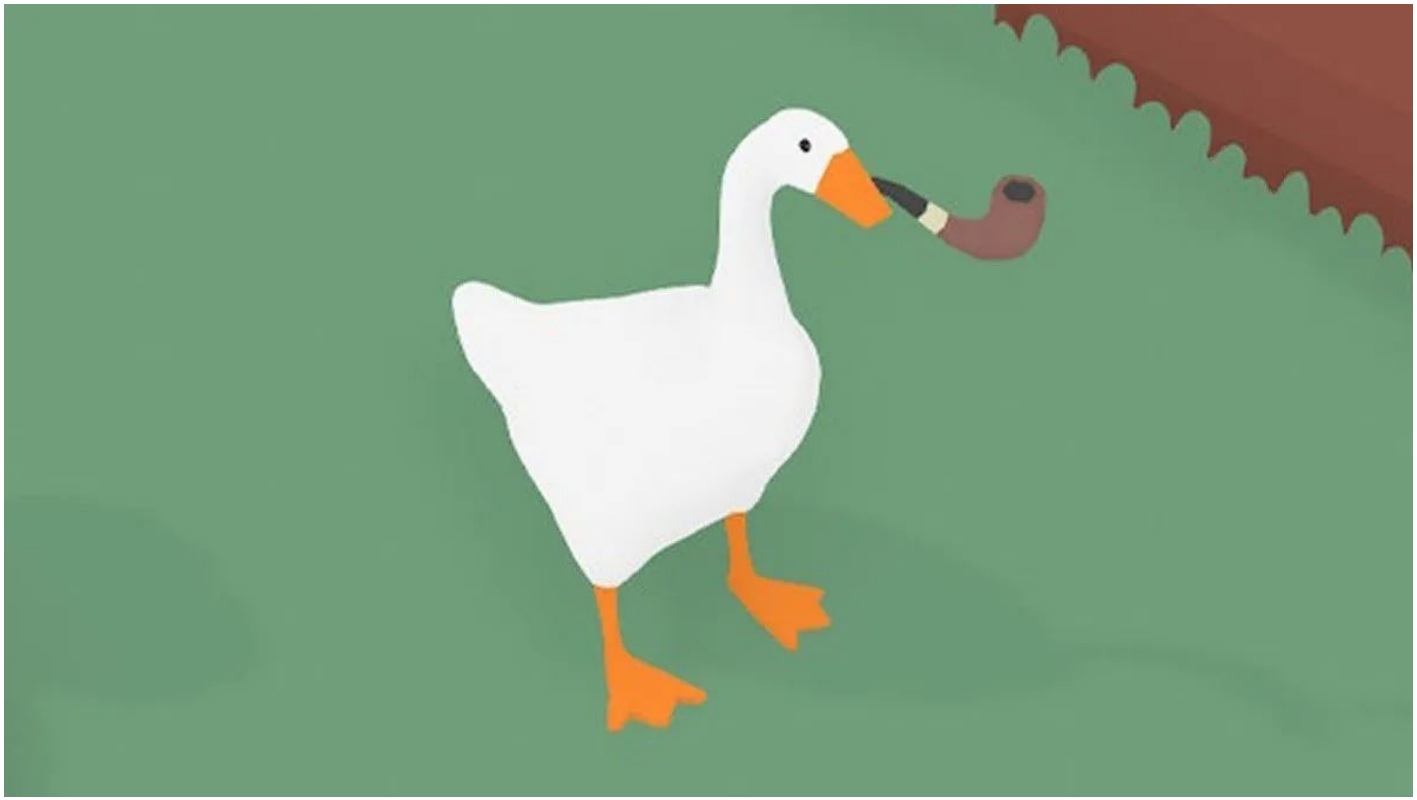
模式协作

运用模式的最佳方式之一，就是让它们走出家门与其他模式互动。模式运用得越多，你就越能在设计中看到它们协同出现。我们为这种能在多种场景下协同工作的模式组合起了个专属名称——**复合模式**，或者**模式之模式**。没错，我们现在讨论的正是由模式组成的超级模式！

- 在同一个设计方案中，模式往往会被联合使用、协同运作。
- **复合模式**，或者**模式之模式** 就是将两种或更多模式整合起来，形成一个能解决常见或通用问题的综合方案。

例子假设

先前有模拟大鹅游戏大受好评，我们公司也准备做一个模拟鸭子的游戏：



例子假设

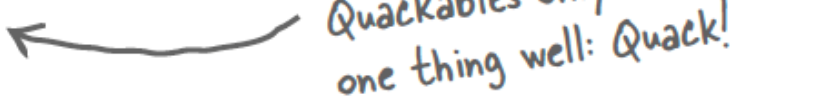
该模拟鸭子的游戏，即鸭子模拟器，需要做到以下这些要求：

- 能够方便地**增加新种类**的鸭子产品，如野鸭、绿头鸭、红头鸭，乃至模型鸭等等；
- 可能会有其他禽类，比如**鹅、大雁**等等也混进来；
- 呱呱叫学家可能会对鸭子的叫声感兴趣；
- 需要对鸭子进行统一装配，以确保他们行为保持一致；
- 还有可能出现鸭子的“家族”，单个鸭子也好、一族鸭子也好都可以进行统一管理；
- 兼具可扩展性与灵活性；

鸭子问题

首先，我们创建一个 Quackable**接口。**
正如之前所说，这次我们要从零开始设计——所有鸭子类都将实现这个 Quackable 接口。这样一来，我们就能明确模拟器中哪些对象会 quack() 方法：比如绿头鸭、红头鸭、野鸭，说不定连橡皮小鸭也会悄悄混进来呢！

```
public interface Quackable {
    public void quack();
}
```



鸭子问题

现在，我们来实现几个支持 Quackable 接口的鸭子类
光有接口没有实现类怎么行？是时候创建一些具体的鸭子类了！

```
public class MallardDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

Your standard Mallard duck.

```
public class RedheadDuck implements Quackable {
    public void quack() {
        System.out.println("Quack");
    }
}
```

We've got to have some variation of species if we want this to be an interesting simulator.

增加其他类型的鸭子

```
public class DuckCall implements Quackable {
    public void quack() {
        System.out.println("Kwak");
    }
}
```

A DuckCall that quacks but doesn't sound quite like the real thing.

```
public class RubberDuck implements Quackable {
    public void quack() {
        System.out.println("Squeak");
    }
}
```

A RubberDuck that makes a squeak when it quacks.

鸭子模拟器

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }

    void simulate() {
        Quackable mallardDuck = new MallardDuck();
        Quackable redheadDuck = new RedheadDuck();
        Quackable duckCall = new DuckCall();
        Quackable rubberDuck = new RubberDuck();

        System.out.println("\nDuck Simulator");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

Here's our main method to get everything going.

We create a simulator and then call its simulate() method.

We need some ducks, so here we create one of each Quackable...

... then we simulate each one.

Here we overload the simulate method to simulate just one duck.

Here we let polymorphism do its magic: no matter what kind of Quackable gets passed in, the simulate() method asks it to quack.

模拟器里突然出现个捣乱的鹅类

```
public class Goose {
    public void honk() {
        System.out.println("Honk");
    }
}
```

A Goose is a honker, not a quacker.

假设我们希望**在任何需要使用鸭子的地方都能使用鹅**——毕竟鹅也会叫、会飞、会游泳，凭什么不能加入模拟器呢？
该用什么模式才能让鹅轻松混入鸭群？

我们需要鹅专用的适配器

由于模拟器需要接收Quackable接口的对象，而鹅并不会呱呱叫（它们是咯咯叫的），这时我们可以通过**适配器模式**，让鹅也能伪装成鸭子。

```
public class GooseAdapter implements Quackable {
    Goose goose;

    public GooseAdapter(Goose goose) {
        this.goose = goose;
    }

    public void quack() {
        goose.honk();
    }
}
```

Remember, an Adapter implements the target interface, which in this case is Quackable.

The constructor takes the goose we are going to adapt.

When quack is called, the call is delegated to the goose's honk() method.

适配器模式实现：让鹅类兼容模拟器

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        simulator.simulate();
    }
    void simulate() {
        Quackable mallardDuck = new MallardDuck();
        Quackable redheadDuck = new RedheadDuck();
        Quackable duckCall = new DuckCall();
        Quackable rubberDuck = new RubberDuck();
        Quackable gooseDuck = new GooseAdapter(new Goose());

        System.out.println("\nDuck Simulator: With Goose Adapter");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

We make a Goose that acts like a Duck by wrapping the Goose in the GooseAdapter.

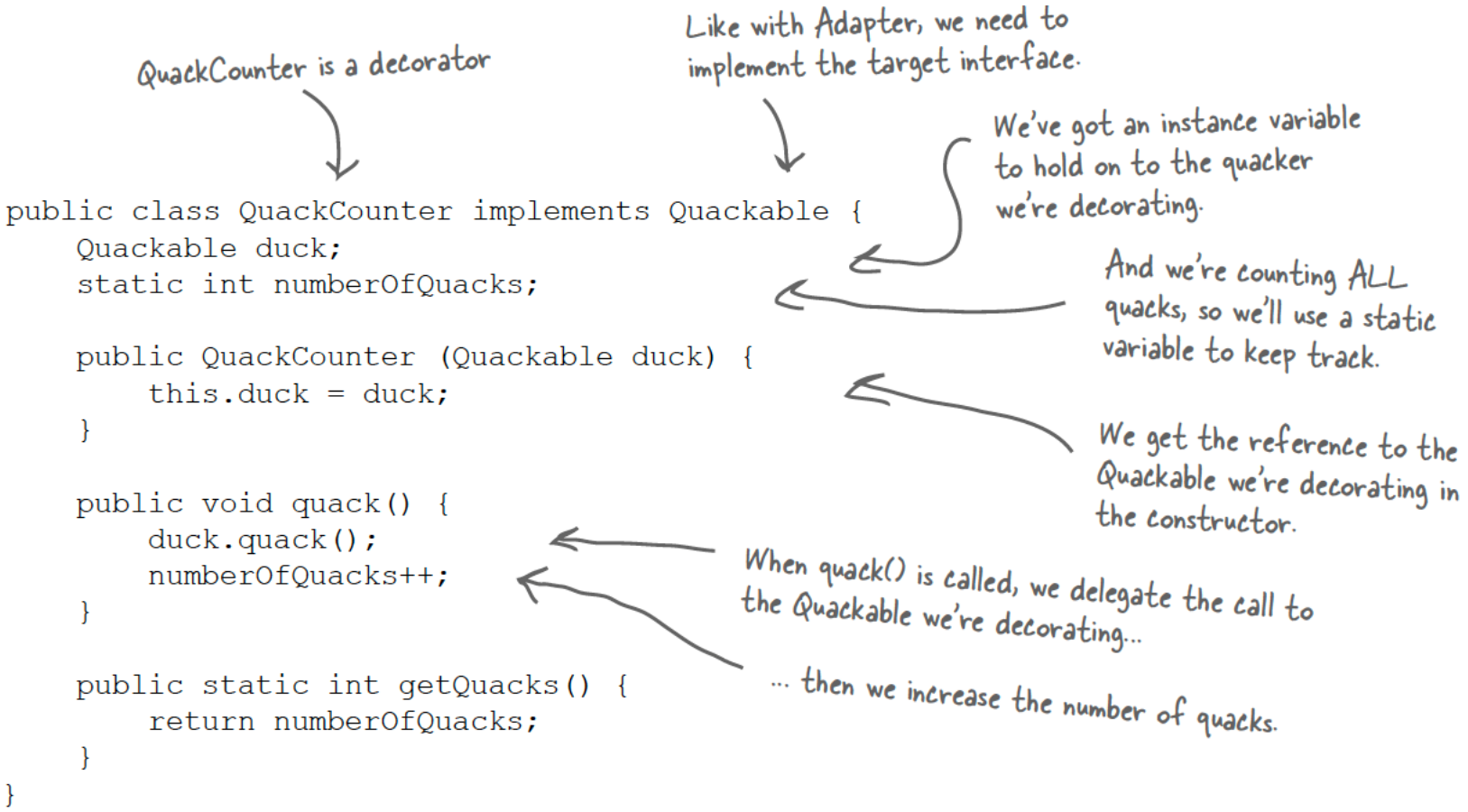
Once the Goose is wrapped, we can treat it just like other duck Quackables.

呱呱叫学家

- 呱呱学家对所有可呱呱叫行为都充满研究热情。他们一直想要统计的，就是鸭群的总叫声次数。
- 问题来了：如何在不修改鸭子类的前提下，实现叫声计数功能?
- 你能想到哪个设计模式可以解决这个问题吗?

叫声计数统计

让我们通过**装饰器模式**来扩展鸭子的行为——用装饰器对象包裹鸭子类，就能**动态添加计数功能。**



必须通过**装饰对象**才能实现功能叠加

- 叫声计数功能确实很棒，让我们发现了这些小家伙们许多不为人知的秘密。不过目前存在一个严重问题——大量叫声未被准确统计。

- 这就是对象包装的痛点所在：**必须确保每个对象都被正确包装，否则就无法获得增强功能。**
- 那么，何不将鸭子的创建过程**集中管理**？换句话说——把鸭子的实例化和装饰逻辑封装起来。
- 这听起来像哪种设计模式？

工厂模式

我们需要建立**质量控制机制**来确保所有鸭子都被正确装饰。为此，我们将专门建造一个**鸭子工厂**来统一生产。这个工厂需要产出包含各类鸭子的**产品族**，因此我们将采用**抽象工厂模式**。

We're defining an abstract factory that subclasses will implement to create different families.

```
public abstract class AbstractDuckFactory {

    public abstract Quackable createMallardDuck();
    public abstract Quackable createRedheadDuck();
    public abstract Quackable createDuckCall();
    public abstract Quackable createRubberDuck();
}
```

Each method creates one kind of duck.

基础工厂：生产未经装饰的鸭子

```
public class DuckFactory extends AbstractDuckFactory {

    public Quackable createMallardDuck() {
        return new MallardDuck();
    }

    public Quackable createRedheadDuck() {
        return new RedheadDuck();
    }

    public Quackable createDuckCall() {
        return new DuckCall();
    }

    public Quackable createRubberDuck() {
        return new RubberDuck();
    }
}
```

DuckFactory extends the abstract factory.

Each method creates a product: a particular kind of Quackable. The actual product is unknown to the simulator – it just knows it's getting a Quackable.

进阶工厂：鸭子叫声计数工厂

CountingDuckFactory also extends the abstract factory.

```
public class CountingDuckFactory extends AbstractDuckFactory {

    public Quackable createMallardDuck() {
        return new QuackCounter(new MallardDuck());
    }

    public Quackable createRedheadDuck() {
        return new QuackCounter(new RedheadDuck());
    }

    public Quackable createDuckCall() {
        return new QuackCounter(new DuckCall());
    }

    public Quackable createRubberDuck() {
        return new QuackCounter(new RubberDuck());
    }
}
```

Each method wraps the Quackable with the quack counting decorator. The simulator will never know the difference; it just gets back a Quackable. But now our rangers can be sure that all quacks ar being counted.

抽象工厂模式如何运作？我们定义一个多态方法：
接收抽象工厂作为参数，调用工厂方法创建对象；这样，只需传入不同的具体工厂，就能在同一个方法中生产不同产品族的对象。

```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();

        simulator.simulate(duckFactory);
    }

    void simulate(AbstractDuckFactory duckFactory) {
        Quackable mallardDuck = duckFactory.createMallardDuck();
        Quackable redheadDuck = duckFactory.createRedheadDuck();
        Quackable duckCall = duckFactory.createDuckCall();
        Quackable rubberDuck = duckFactory.createRubberDuck();
        Quackable gooseDuck = new GooseAdapter(new Goose());

        System.out.println("\nDuck Simulator: With Abstract Factory");

        simulate(mallardDuck);
        simulate(redheadDuck);
        simulate(duckCall);
        simulate(rubberDuck);
        simulate(gooseDuck);

        System.out.println("The ducks quacked " +
                           QuackCounter.getQuacks() +
                           " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

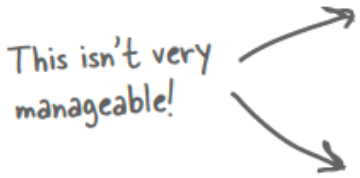
First we create the factory that we're going to pass into the simulate() method.

The simulate() method takes an AbstractDuckFactory and uses it to create ducks rather than instantiating them directly.

Nothing changes here! Same ol' code.

鸭群管理

- 随着鸭子种类不断增加，单独管理确实变得棘手。有没有办法能让我们**统一管理**所有鸭子，甚至可以追踪特定的几个鸭子**族群**呢？




```
Quackable mallardDuck = duckFactory.createMallardDuck();
Quackable redheadDuck = duckFactory.createRedheadDuck();
Quackable duckCall = duckFactory.createDuckCall();
Quackable rubberDuck = duckFactory.createRubberDuck();
Quackable gooseDuck = new GooseAdapter(new Goose());

simulate(mallardDuck);
simulate(redheadDuck);
simulate(duckCall);
simulate(rubberDuck);
simulate(gooseDuck);
```

鸭群管理

- 我们需要一种能够**统一管理**鸭群集合（包括子集合，以满足呱呱叫学家的族群追踪需求）的方案，最好还能**批量执行操作**。

- 哪个设计模式能实现这些功能?

鸭群管理

Remember the Composite Pattern that allows us to treat a collection of objects in the same way as individual objects? What better composite than a flock of Quackables!

Remember, the composite needs to implement the same interface as the leaf elements. Our leaf elements are Quackables.

```
public class Flock implements Quackable {
    ArrayList quackers = new ArrayList();

    public void add(Quackable quacker) {
        quackers.add(quacker);
    }

    public void quack() {
        Iterator iterator = quackers.iterator();
        while (iterator.hasNext()) {
            Quackable quacker = (Quackable)iterator.next();
            quacker.quack();
        }
    }
}
```

We're using an ArrayList inside each Flock to hold the Quackables that belong to the Flock.

The add() method adds a Quackable to the Flock.

There it is! The Iterator Pattern at work!

Now for the quack() method – after all, the Flock is a Quackable too. The quack() method in Flock needs to work over the entire Flock. Here we iterate through the ArrayList and call quack() on each element.

组合模式已就绪

```
public class DuckSimulator {
    // main method here

    void simulate(AbstractDuckFactory duckFactory) {
        Quackable redheadDuck = duckFactory.createRedheadDuck();
        Quackable duckCall = duckFactory.createDuckCall();
        Quackable rubberDuck = duckFactory.createRubberDuck();
        Quackable gooseDuck = new GooseAdapter(new Goose());
        System.out.println("\nDuck Simulator: With Composite - Flocks");

        Flock flockOfDucks = new Flock();

        flockOfDucks.add(redheadDuck);
        flockOfDucks.add(duckCall);
        flockOfDucks.add(rubberDuck);
        flockOfDucks.add(gooseDuck);

        Flock flockOfMallards = new Flock();
```

Create all the Quackables, just like before.

First we create a Flock, and load it up with Quackables.

Then we create a new Flock of Mallards.

安全性与透明性

- 是否还记得？在组合模式章节中，组合节点（如菜单Menu）和叶节点（如菜单项MenuItem）拥有完全相同的方法集（包括add()方法）。由于方法集完全一致，我们甚至可以调用那些对叶节点无实际意义的方法（例如试图通过add()方法向菜单项添加子项）。这种设计的优势在于：**透明性**：客户端无需区分处理的是叶节点还是组合节点，**统一性**：对两者调用相同方法即可。
- 而在当前实现中，我们决定将组合节点的子节点管理方法与叶节点分离：仅**鸭群（Flock）**拥有add()方法，而明确禁止向**单个鸭子（Duck）**添加子项的设计限制。这种设计带来两个影响：一是**安全性提升**，客户端无法调用组件不支持的方法，二是**透明性降低**，客户端必须明确知道某个Quackable是Flock才能添加成员。

同时支持个体鸭子追踪

- 组合模式运行完美！不过现在有个新需求：我们还需要实时追踪单只鸭子的叫声。
- 是否也有相应的实现方案?

观察者模式

First we need an Observable interface.

Remember that an Observable is the object being observed. An Observable needs methods for registering and notifying observers. We could also have a method for removing observers, but we'll keep the implementation simple here and leave that out.

QuackObservable is the interface that Quackables should implement if they want to be observed.

```
public interface QuackObservable {
    public void registerObserver(Observer observer);
    public void notifyObservers();
}
```

It also has a method for notifying the observers.

It has a method for registering Observers. Any object implementing the Observer interface can listen to quacks. We'll define the Observer interface in a sec.

Now we need to make sure all Quackables implement this interface...

```
public interface Quackable extends QuackObservable {
    public void quack();
}
```

So, we extend the Quackable interface with QuackObserver.

QuackObservable

当前需要确保所有实现Quackable**接口的具体类都能支持观察者功能，即**QuackObservable**。**
通常做法是在每个类中重复实现注册和通知逻辑，但这次我们决定采用更优雅的设计：将核心的注册和通知机制封装到专门的Observable辅助类中，再通过组合方式让QuackObservable接口的实现类委托调用这些功能。这种架构只需在Observable类中编写一次核心逻辑代码，各个Quackable实现类仅需保留最小化的委托调用代码即可获得完整的观察者能力，既避免了代码重复，又实现了关注点分离，还能确保后续新增的Quackable类都能快速获得观察者功能支持。

Observable implements all the functionality a Quackable needs to be an observable. We just need to plug it into a class and have that class delegate to Observable.

Observable must implement QuackObservable because these are the same method calls that are going to be delegated to it.

```
public class Observable implements QuackObservable {
    ArrayList observers = new ArrayList();
    QuackObservable duck;

    public Observable(QuackObservable duck) {
        this.duck = duck;
    }

    public void registerObserver(Observer observer) {
        observers.add(observer);
    }

    public void notifyObservers() {
        Iterator iterator = observers.iterator();
        while (iterator.hasNext()) {
            Observer observer = (Observer)iterator.next();
            observer.update(duck);
        }
    }
}
```

In the constructor we get passed the QuackObservable that is using this object to manage its observable behavior. Check out the notify() method below; you'll see that when a notify occurs, Observable passes this object along so that the observer knows which object is quacking.

Here's the code for registering an observer.

And the code for doing the notifications.

将辅助类 Observable 的功能整合到 Quackable 各类实现中

All we need to do is make sure the Quackable classes
are composed with an Observable and that they know how to delegate to it.

```
public class MallardDuck implements Quackable {
    Observable observable;

    public MallardDuck() {
        observable = new Observable(this);
    }

    public void quack() {
        System.out.println("Quack");
        notifyObservers();
    }

    public void registerObserver(Observer observer) {
        observable.registerObserver(observer);
    }

    public void notifyObservers() {
        observable.notifyObservers();
    }
}
```

Each Quackable has an Observable instance variable.

In the constructor, we create an Observable and pass it a reference to the MallardDuck object.

When we quack, we need to let the observers know about it.

Here's our two QuackObservable methods. Notice that we just delegate to the helper.

观察者模式

我们只需确保所有 Quackable 实现类都**组合**了一个 Observable 对象，并正确实现**委托调用**即可。

```
public interface Observer {
    public void update(QuackObservable duck);
}
```

观察者模式

We need to implement the Observable interface or else we won't be able to register with a QuackObservable.

```
public class Quackologist implements Observer {

    public void update(QuackObservable duck) {
        System.out.println("Quackologist: " + duck + " just quacked.");
    }
}
```

The Quackologist is simple; it just has one method, update(), which prints out the Quackable that just quacked.

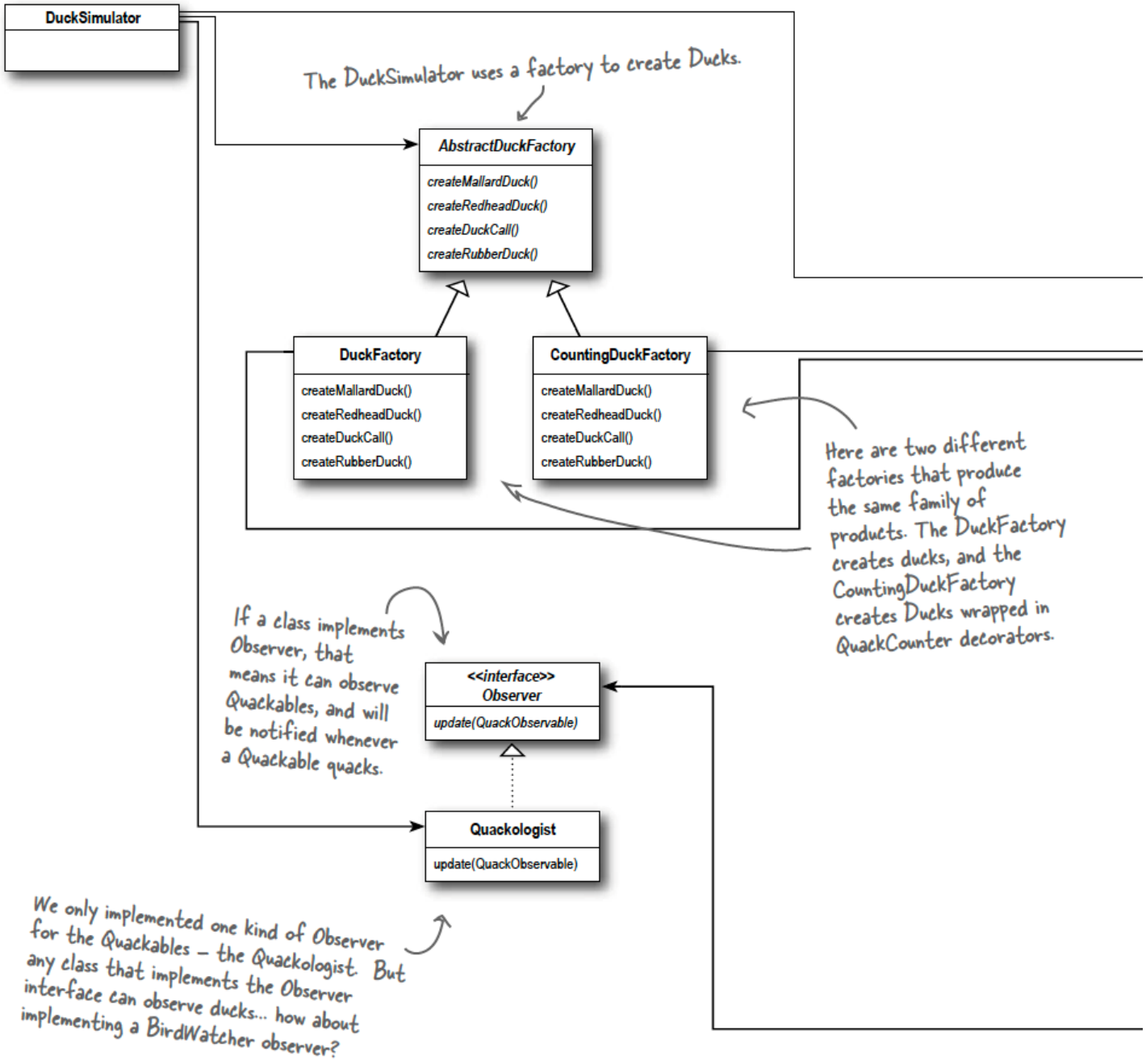
最终的鸭子模拟器

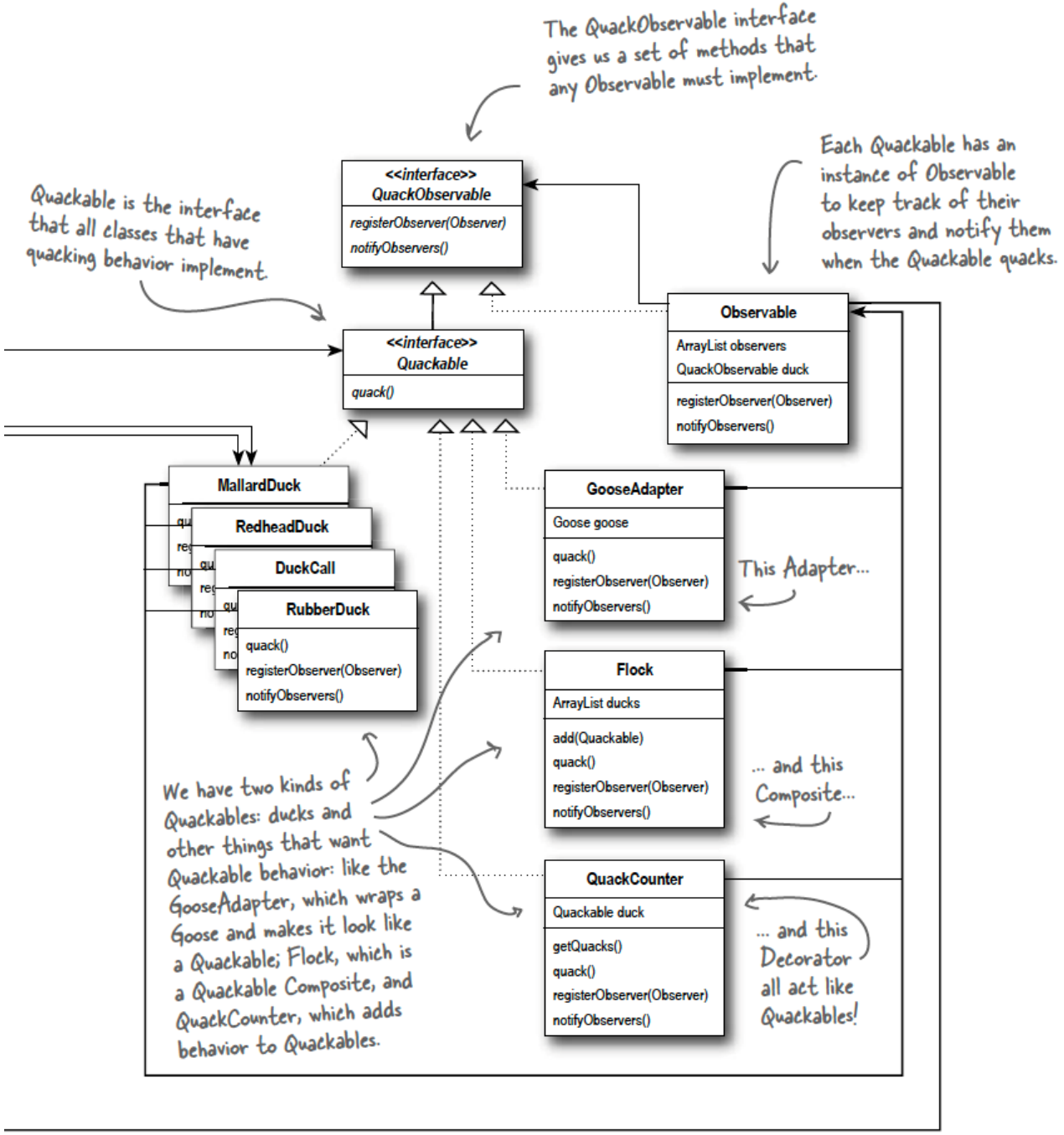
```
public class DuckSimulator {
    public static void main(String[] args) {
        DuckSimulator simulator = new DuckSimulator();
        AbstractDuckFactory duckFactory = new CountingDuckFactory();

        simulator.simulate(duckFactory);
    }

    void simulate(AbstractDuckFactory duckFactory) {

        // create duck factories and ducks here

        // create flocks here

        System.out.println("\nDuck Simulator: With Observer");
        Quackologist quackologist = new Quackologist();
        flockOfDucks.registerObserver(quackologist);

        simulate(flockOfDucks);

        System.out.println("\nThe ducks quacked " +
                           QuackCounter.getQuacks() +
                           " times");
    }

    void simulate(Quackable duck) {
        duck.quack();
    }
}
```

All we do here is create a Quackologist and set him as an observer of the flock.

This time we'll we just simulate the entire flock.

Let's give it a try and see how it works!


DuckSimulator
The DuckSimulator uses a factory to create Ducks.
AbstractDuckFactory
createMallardDuck()
createRedheadDuck()
createDuckCall()
createRubberDuck()
DuckFactory
createMallardDuck()
createRedheadDuck()
createDuckCall()
createRubberDuck()
CountingDuckFactory
createMallardDuck()
createRedheadDuck()
createDuckCall()
createRubberDuck()
Here are two different factories that produce the same family of products. The DuckFactory creates ducks, and the CountingDuckFactory creates Ducks wrapped in QuackCounter decorators.
If a class implements Observer, that means it can observe Quackables, and will be notified whenever a Quackable quacks.
<<interface>>
Observer
update(QuackObservable)
Quackologist
update(QuackObservable)
We only implemented one kind of Observer for the Quackables – the Quackologist. But any class that implements the Observer interface can observe ducks... how about implementing a BirdWatcher observer?


The QuackObservable interface gives us a set of methods that any Observable must implement.
Each Quackable has an instance of Observable to keep track of their observers and notify them when the Quackable quacks.
Quackable is the interface that all classes that have quacking behavior implement.
<<interface>>
QuackObservable
registerObserver(Observer)
notifyObservers()
<<interface>>
Quackable
quack()
Observable
ArrayList observers
QuackObservable duck
registerObserver(Observer)
notifyObservers()
MallardDuck
RedheadDuck
DuckCall
RubberDuck
quack()
registerObserver(Observer)
notifyObservers()
GooseAdapter
Goose goose
quack()
registerObserver(Observer)
notifyObservers()
This Adapter...
Flock
ArrayList ducks
add(Quackable)
quack()
registerObserver(Observer)
notifyObservers()
... and this Composite...
QuackCounter
Quackable duck
getQuacks()
quack()
registerObserver(Observer)
notifyObservers()
... and this Decorator all act like Quackables!
We have two kinds of Quackables: ducks and other things that want Quackable behavior: like the GooseAdapter, which wraps a Goose and makes it look like a Quackable; Flock, which is a Quackable Composite, and QuackCounter, which adds behavior to Quackables.